Session 1 said *what* the pipeline is. This session is about **what lives at each seam**, what Spring gives you out of the box, and why the design is shaped this way at all.

1. The four seams at a glance
2. One request, traced through all four
3. `HandlerMapping` — contract, hierarchy, implementations, ordering
4. `HandlerAdapter` — contract, the Adapter pattern, implementations, what lives inside `RequestMappingHandlerAdapter`
5. **The handler** — the whole Controller family, class by class
6. `ViewResolver` **and** `View` — two interfaces, not one; the resolver chain, and what a JSP view actually does
7. Why it is structured like this — and what it costs
8. The cheat sheet: handler type → mapping → adapter
9. Demo project + exercises

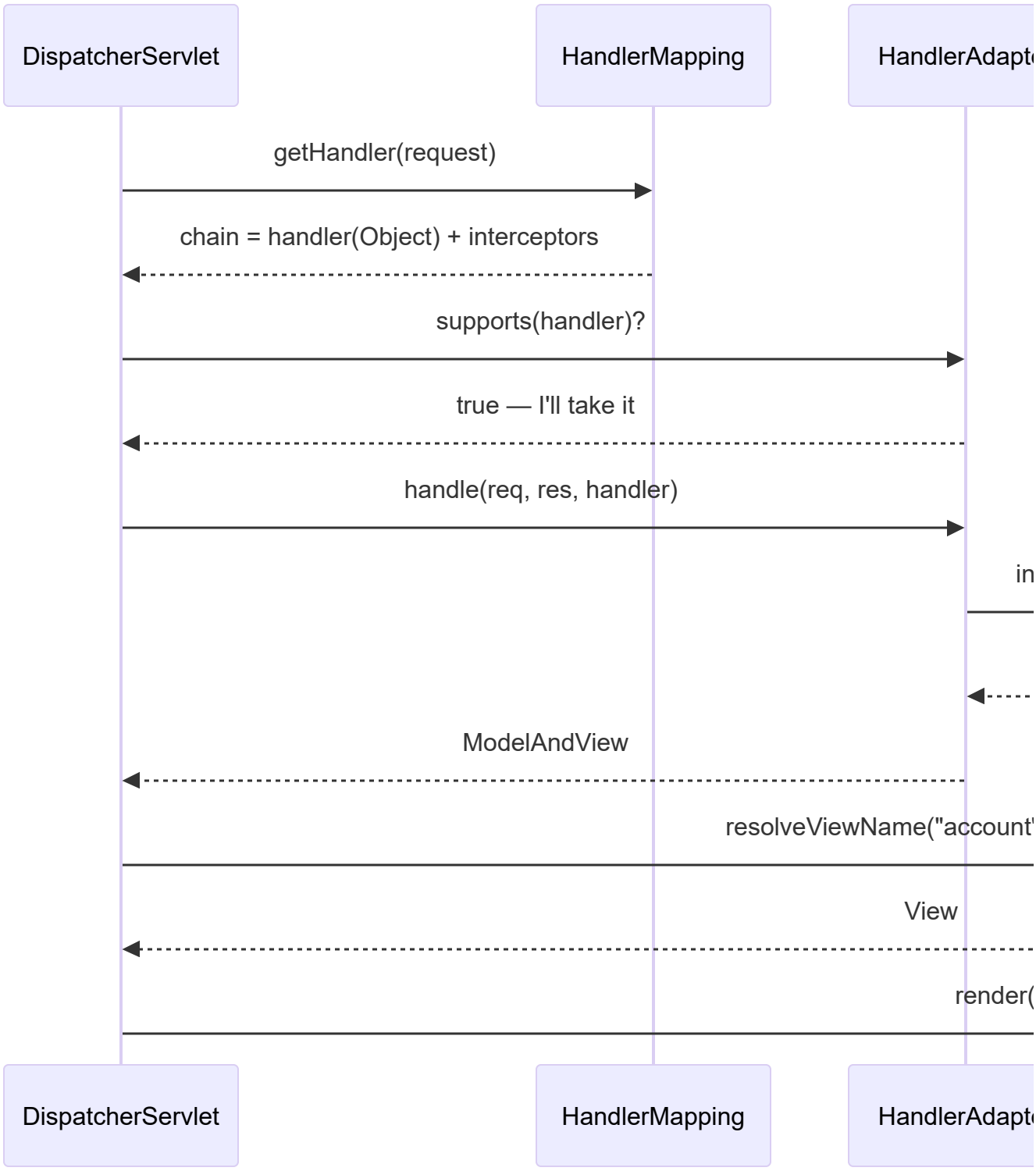
The Same Thing As A Table

#	Seam	The question it answers	Return type
1	`HandlerMapping`	**Which** handler serves this URL?	`HandlerExecutionChain`
2	`HandlerAdapter`	**How** do I invoke that handler?	`ModelAndView` or `null`
3	the handler	**What** is the answer?	model + logical view name
4	`ViewResolver` → `View`	**Which page**, rendered **how**?	bytes on the wire

Four interfaces. `DispatcherServlet` holds a `List` of the first two and the fourth, chosen at startup from your beans. It holds **no** reference to your controllers at all.

One Request, All Four Seams

`GET /accounts?id=5`



Read the two `-->>` arrows back into `DS` on lines 2 and 8. **Everything returns to the hub.** The components never call each other directly — that is what keeps them independent.

---

1 · HandlerMapping

The "which handler?" seam

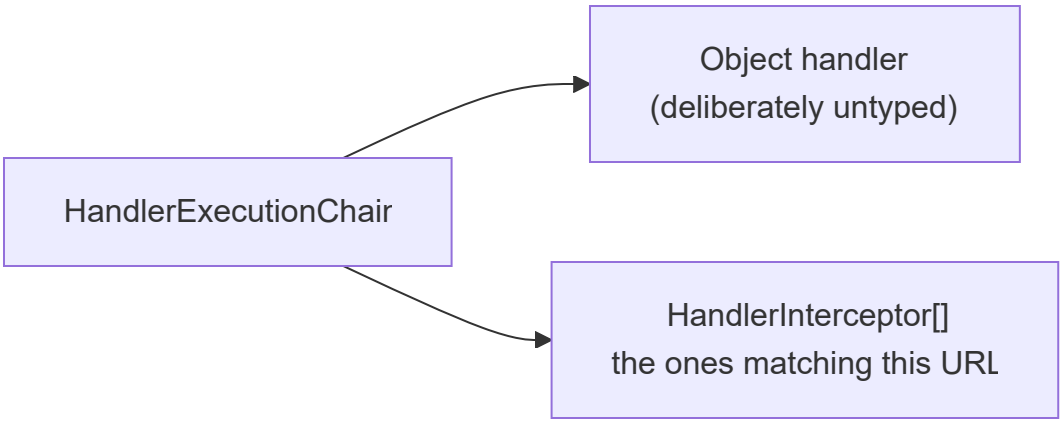
`HandlerMapping` — The Contract

```
public interface HandlerMapping {

    @Nullable
    HandlerExecutionChain getHandler(HttpServletRequest request) throws
Exception;
}
```

One method. Two things to notice, and both matter:

- it returns `null` when it doesn't recognise the URL — that is the signal for `DispatcherServlet` to try the next mapping
- it does **not** return a handler; it returns a `HandlerExecutionChain`



Interceptors are **not** a separate lifecycle stage that `DispatcherServlet` manages on its own. They arrive bundled with the handler, from the mapping. That is why you configure them on the mapping bean.

Why The Handler Is `Object`

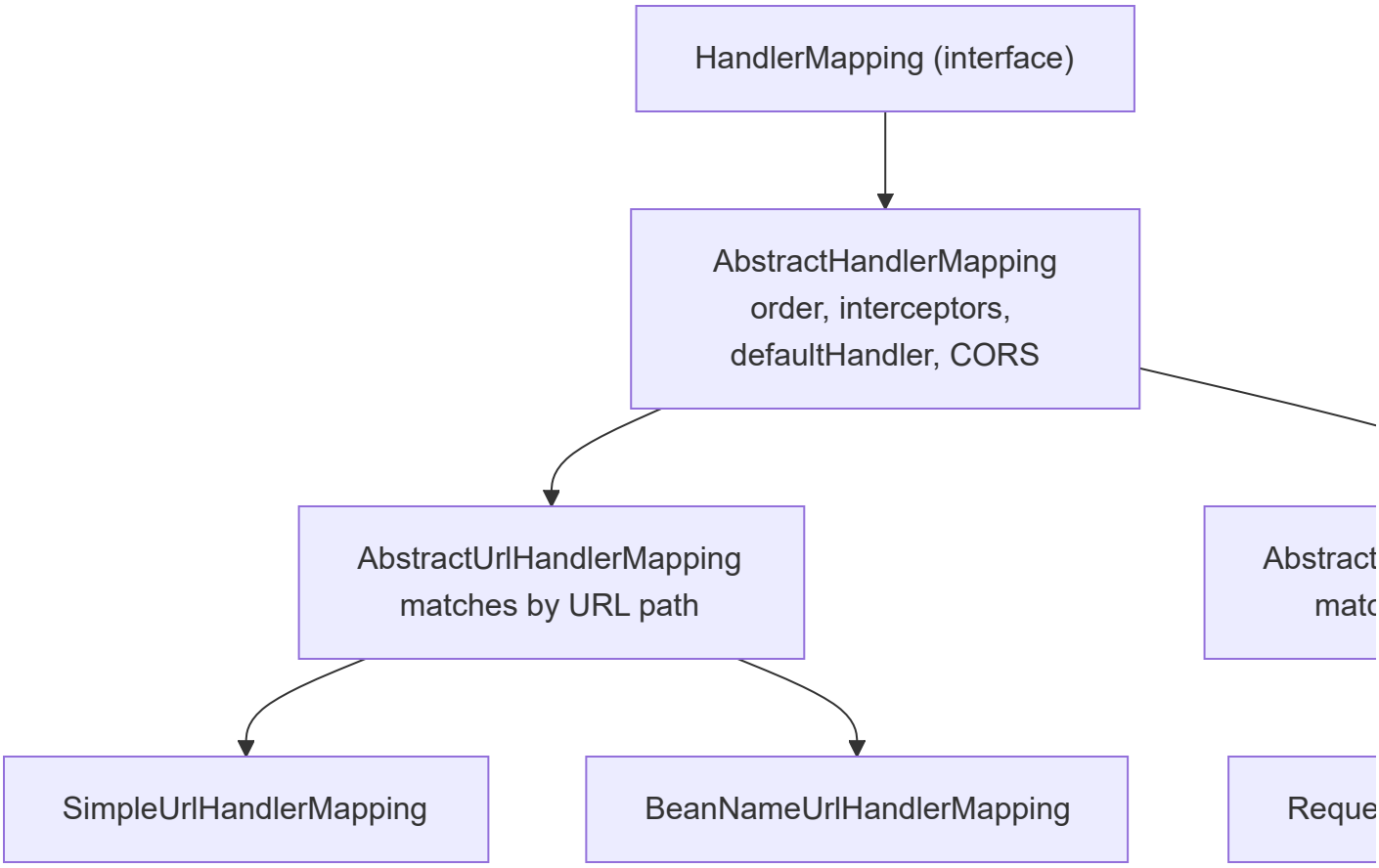
This is the single most important line in the whole framework:

```
Object handler = mappedHandler.getHandler();   // NOT Controller. Object.
```

If that type were `Controller`, then every handler in Spring MVC forever would have had to implement `Controller`. Annotated methods, functional endpoints and static-resource handlers would all have been impossible without rewriting `DispatcherServlet`.

Typing it as `Object` costs one thing — `DispatcherServlet` now cannot call it — and seam 2 exists purely to pay that cost.

`HandlerMapping` — The Class Hierarchy



The split at the second level is the real story: **URL → bean** on the left, **URL → a specific method on a bean** on the right. Everything else follows from that.

`RequestMappingHandlerMapping` — The One You Use

Maps to a **method**, not a class.

```
@Controller
public class AccountController {
    @GetMapping("/accounts")           //  <- this is the mapping
    public String list(@RequestParam int id, Model model) { ... }
}
```

- It scans every bean in the context for `@Controller` or `@RequestMapping` **at type level** — component scanning is not required, a plain `<bean>` declaration is enough
- It builds a `RequestMappingInfo` per method: path **plus** method, params, headers, consumes, produces
- The handler it returns is a `HandlerMethod` — a bean instance plus a `java.lang.reflect.Method`
- Registered for you by `@EnableWebMvc` / `<mvc:annotation-driven/>`

Because it matches on more than the path, two methods can share a URL and differ only by `produces` or by HTTP method. No other mapping can do that.

`SimpleUrlHandlerMapping` — The Explicit Map

```
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
        <props>
            <prop key="/home">homeController</prop>
            <prop key="/reports/**">reportController</prop>
        </props>
    </property>
    <property name="order" value="0"/>
</bean>
```

No annotations, no conventions — the routing table is a literal map you wrote.

Still very much alive in modern Spring: `@EnableWebMvc` uses a `SimpleUrlHandlerMapping` internally for **static resources** (`addResourceHandlers`) and for **view controllers** (`addViewControllers`). You just don't normally see the bean.

`BeanNameUrlHandlerMapping` — Routing By Bean Name

The URL **is** the bean name.

```
<bean name="/legacy" class="com.example.LegacyController"/>
```

`GET /legacy` → that bean. Any bean whose name starts with `/` becomes a route.

Two details worth knowing:

- it must be `name=`, not `id=` — XML `id` is an XML `ID` type and cannot contain `/`
- it is one of the two defaults in `DispatcherServlet.properties`, so it is active in a bare `DispatcherServlet` with no config at all

You will meet this in old codebases. You will not choose it for new ones.

Several Mappings At Once — Chain Of Responsibility

`DispatcherServlet` keeps a `List<HandlerMapping>`, sorted by `Ordered`, and asks each in turn:

```
for (HandlerMapping mapping : this.handlerMappings) {
    HandlerExecutionChain handler = mapping.getHandler(request);
    if (handler != null) {
        return handler;          // first non-null wins
    }
}
return null;                     // nobody claimed it → 404 path
```

`AbstractHandlerMapping` — The Knobs You Inherit

Every mapping gets these, whichever subclass you use:

Property	What it does
`order`	position in the chain above
`interceptors`	the `HandlerInterceptor`s bundled into every chain this mapping returns
`defaultHandler`	used when nothing matched — turns a 404 into a fallback page
`alwaysUseFullPath`	match against the full path instead of the path within the servlet mapping
`corsConfigurations`	per-pattern CORS rules, applied before the handler runs

`HandlerInterceptor` — Delivered By The Mapping

```
public interface HandlerInterceptor {
    default boolean preHandle(HttpServletRequest req, HttpServletResponse res,
Object handler);
    default void postHandle(HttpServletRequest req, HttpServletResponse res,
                            Object handler, @Nullable ModelAndView mav);
    default void afterCompletion(HttpServletRequest req, HttpServletResponse
res,
                                 Object handler, @Nullable Exception ex);
}
```

Hook	Fires	Use it for
`preHandle`	before the adapter is even chosen	auth, timing start — `false` **aborts the request**
`postHandle`	handler done, view **not yet** rendered	last chance to add a model attribute
`afterCompletion`	after rendering, **even on exception**	cleanup, total timing

Note `handler` is `Object` here too. To inspect a method's annotations you must check `instanceof HandlerMethod` first — which only holds for annotated controllers.

---

2 · HandlerAdapter

The "how do I call it?" seam

`HandlerAdapter` — The Contract

```
public interface HandlerAdapter {

    boolean supports(Object handler);

    @Nullable
    ModelAndView handle(HttpServletRequest request,
                        HttpServletResponse response,
                        Object handler) throws Exception;
```

```
    // getLastModified(...) — deprecated since 5.3.9, ignore it
}
```

The selection loop is as dumb as it looks:

```
for (HandlerAdapter adapter : this.handlerAdapters) {
    if (adapter.supports(handler)) {
        return adapter;                 // first yes wins
    }
}
throw new ServletException("No adapter for handler [" + handler + "]");
```

That exception message is one you will eventually see. It means: a mapping found your handler, but **no adapter knew how to invoke it.**

Why This Seam Has To Exist

Two handlers, both perfectly valid, with **incompatible** signatures:

```
// shape A — a method, any name, any parameters, many return types
@GetMapping("/accounts")
public String list(@RequestParam int id, Model model) { ... }
```

```
// shape B — a fixed interface method
public class LegacyController implements Controller {
    public ModelAndView handleRequest(HttpServletRequest req,
HttpServletResponse res) { ... }
}
```

There is no single Java call that invokes both. `DispatcherServlet` needs a translator per shape — and that is the textbook definition of the **Adapter** pattern.

`HandlerAdapter` — The Implementations

Adapter	Invokes	Notes
`RequestMappingHandlerAdapter`	`HandlerMethod`	@RequestMapping

Adapter	Invokes	Notes
`SimpleControllerHandlerAdapter`	`Controller`	calls `handleRequest(req, res)`.
`HttpRequestHandlerAdapter`	`HttpRequestHandler`	calls `handleRequest(req, res)`
`SimpleServletHandlerAdapter`	`jakarta.servlet.Servlet`	lets a raw servlet be a handler. Not registered by default

Three of these are defaults in `DispatcherServlet.properties`: `HttpRequestHandlerAdapter`, `SimpleControllerHandlerAdapter`, `RequestMappingHandlerAdapter`.

Writing Your Own Adapter

Rare in production, but the fastest way to prove the design to yourself:

```
public class GreetingHandlerAdapter implements HandlerAdapter {

    @Override
    public boolean supports(Object handler) {
        return handler instanceof GreetingHandler;      // your own handler
type
    }

    @Override
    public ModelAndView handle(HttpServletRequest req, HttpServletResponse
res, Object handler) {
        String name = req.getParameter("name");
        String text = ((GreetingHandler) handler).greet(name == null ? "World"
: name);
        return new ModelAndView("greet", "message", text);
    }
}
```

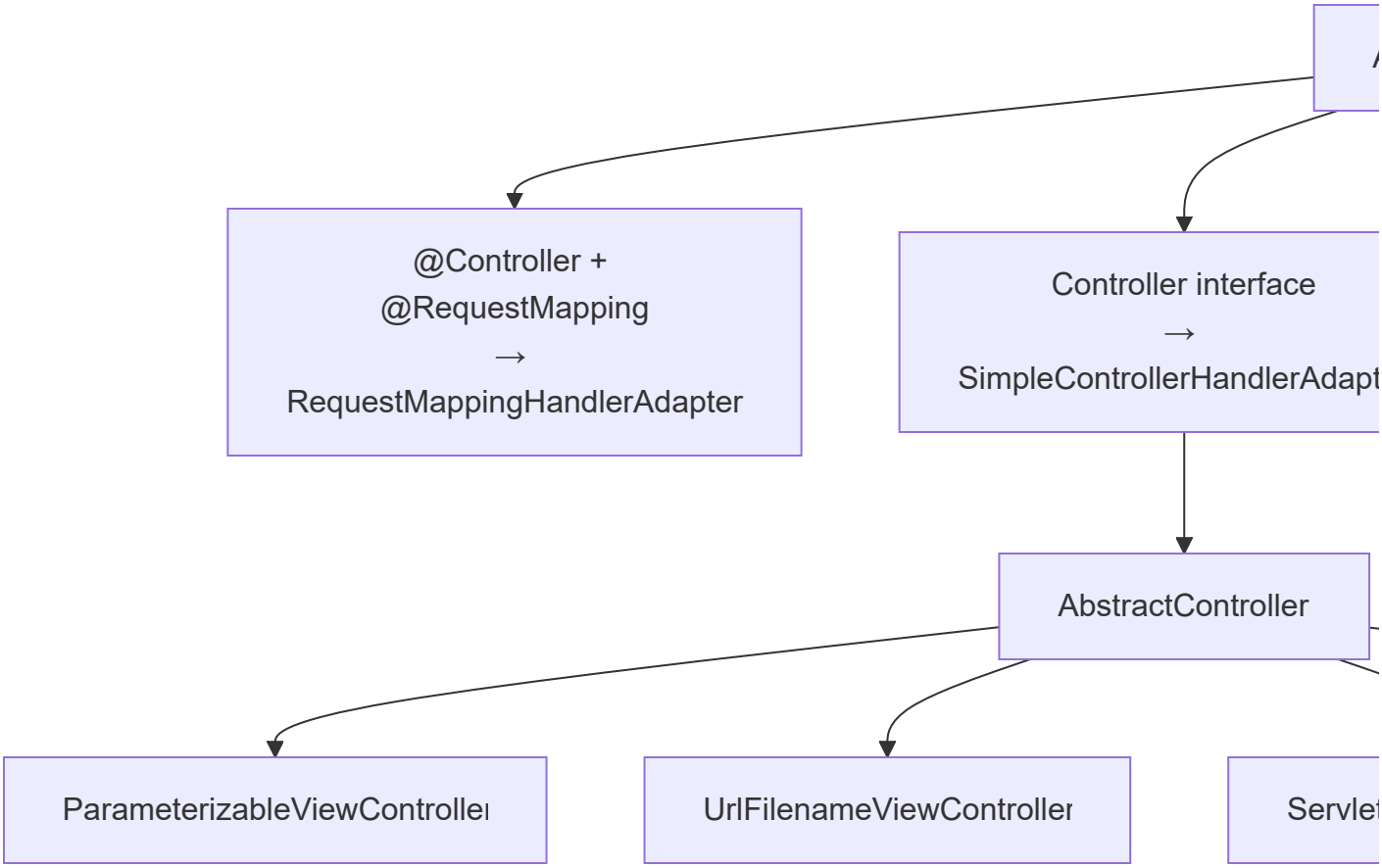
`GreetingHandler` is an interface **you** invented. Spring has never heard of it, and it now works end to end. That is what "the handler is `Object`" actually buys.

---

3 · The Handler

Every shape Spring will accept

The Controller Family Tree



Four top-level shapes, four adapters. The correspondence is exact, and it is not a coincidence.

`Controller` — The Original Interface

```
@FunctionalInterface
public interface Controller {
    @Nullable
    ModelAndView handleRequest(HttpServletRequest request,
                               HttpServletResponse response) throws Exception;
}
```

```
public class LegacyController implements Controller {
    public ModelAndView handleRequest(HttpServletRequest req,
HttpServletResponse res) {
        ModelAndView mav = new ModelAndView("account");
        mav.addObject("account",
service.find(Integer.parseInt(req.getParameter("id"))));
        return mav;
    }
}
```

One class, one URL. Manual `getParameter`, manual `parseInt`, manual null handling. **This is the "before" picture** — it is precisely the boilerplate `RequestMappingHandlerAdapter` was built to delete.

Returning `null` is legal and means *"I already wrote the response, skip view resolution."*

`AbstractController` — The Convenience Base

```
public class AccountController extends AbstractController {

    public AccountController() {
        setSupportedMethods("GET");        // 405 for anything else
        setRequireSession(true);           // 500 if there is no HttpSession
        setCacheSeconds(60);               // writes Cache-Control headers
    }

    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
HttpServletResponse res) {
        return new ModelAndView("account");
    }
}
```

It implements `handleRequest`, runs those checks, then calls your `handleRequestInternal`. Template Method pattern, inside a Strategy, invoked through an Adapter.

The options come from its parent `WebContentGenerator`, which is also the parent of the resource handler — which is why static resources get cache headers the same way.

`ParameterizableViewController` — A Handler With No Code

Sometimes a URL just needs to show a page. There is no logic to write.

```
<bean id="aboutController"

class="org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="about"/>
</bean>
```

That's the whole handler. You have written zero Java.

The Rest Of The `AbstractController` Branch

Class	What it does	When you'd meet it
`UrlFilenameViewController`	derives the view name from the URL — `/help/index.html` → view `index`	content-only sites with no logic
`ServletForwardingController`	forwards to a servlet declared in `web.xml` by its **servlet name**	putting Spring MVC in front of an old servlet without rewriting it
`ServletWrappingController`	instantiates and wraps a servlet class itself, no `web.xml` entry needed	the same, but self-contained

`ServletForwardingController` and `ServletWrappingController` exist for one reason: **incremental migration.** You can put a `DispatcherServlet` at `/` over a legacy app and forward the URLs you haven't converted yet.

`HttpRequestHandler` — Raw And Fast

```
@FunctionalInterface
public interface HttpRequestHandler {
    void handleRequest(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException;
}
```

No `ModelAndView`. No model. No view. You write to the response and you are done — a servlet's `service()` method with Spring dependency injection available.

The implementation you use every day without noticing is `ResourceHttpRequestHandler`: the thing that actually serves your CSS, JS and images when you call `addResourceHandlers`. It needs no view resolution, so it uses the shape that has none.

Also here: `HttpInvokerServiceExporter` and friends, for remoting.

`@Controller` — The Modern Shape

```
@Controller
public class AccountController {

    private final AccountService service;
    public AccountController(AccountService service) { this.service = service;
}

    @GetMapping("/accounts")
    public String list(@RequestParam int id, Model model) {
        model.addAttribute("account", service.find(id));
        return "account";
    }

    @PostMapping("/accounts")
    @ResponseBody
    public Account create(@Valid @RequestBody Account account) { ... }
}
```

- **many URLs per class**, one method each
- any method name, any parameter list, many return types
- `@RestController` = `@Controller` + `@ResponseBody` on every method

None of this flexibility lives in the annotation. It lives in `RequestMappingHandlerAdapter` and its resolver lists. The annotations are just metadata that `RequestMappingHandlerMapping` reads.

`Model`, `ModelMap`, `ModelAndView`

Type	View name comes from	Notes
`Model`	the returned `String`	the normal choice
`ModelMap`	the returned `String`	same thing with a `Map` API
`ModelAndView`	inside the object	needed when the view name is computed conditionally, and the **only** option for legacy shapes

Underneath it is a `Map<String, Object>`. `model.addAttribute("account", acc)` becomes a request attribute, which the JSP reads as `${account}`.

---

4 · ViewResolver and View

The "which page, rendered how?" seam

Two Interfaces, Not One

This trips people up. There are **two** steps here, with two different jobs.

```
public interface ViewResolver {
    @Nullable
    View resolveViewName(String viewName, Locale locale) throws Exception;
}
```

```
public interface View {
    @Nullable default String getContentType() { return null; }

    void render(@Nullable Map<String, ?> model,
                HttpServletRequest request,
                HttpServletResponse response) throws Exception;
}
```

Step	Job	Runs
`ViewResolver`	name → object. **Lookup only.** Writes nothing	once per request

Step	Job	Runs
`View`	object → bytes. **Rendering only.** Knows nothing about names	once per request

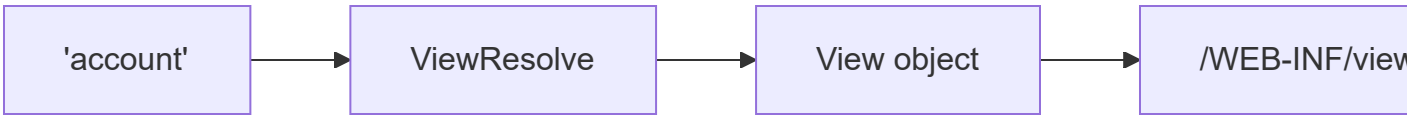
Splitting them means resolvers are cacheable and views are reusable. A `View` bean can be shared across requests precisely because it holds no request state.

Why Not Just Return The JSP Path?

Because the controller would then know it is talking to a JSP.

```
return "account";                          // ✅ logical
return "/WEB-INF/views/account.jsp";       // ❌ now the controller owns the view technology
```

Swap JSP for Thymeleaf, or add a PDF export, and the first version needs **one config change**. The second needs you to edit every controller in the codebase.



`ViewResolver` — The Implementations

Implementation	Resolves a name to	Status
`InternalResourceViewResolver`	a JSP, via `prefix` + `suffix`	**the default choice**
`UrlBasedViewResolver`	its parent — pure convention, any `viewClass`	fine to use directly
`BeanNameViewResolver`	a `View` **bean** whose name equals the view name	excellent for one-off custom views
`ContentNegotiatingViewResolver`	delegates to other resolvers by `Accept` header	HTML *and* JSON from one URL

Implementation	Resolves a name to	Status
`ResourceBundleViewResolver`	entries in a `.properties` file	**deprecated since 5.3**
`XmlViewResolver`	`View` beans in a separate XML file	**deprecated since 5.3**
`FreeMarkerViewResolver`, `ThymeleafViewResolver`	template engines	`UrlBasedViewResolver` subclasses

`InternalResourceViewResolver`

```
<bean
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/views/"/>
    <property name="suffix" value=".jsp"/>
</bean>
```

`"account"` → `/WEB-INF/views/account.jsp`, wrapped in an `InternalResourceView` that does a `RequestDispatcher.forward`.

Three things about it:

- it **always** resolves. It never returns `null`, because any string concatenates into a path — so it must be **last** in the chain, always
- it picks `JstlView` automatically if JSTL is on the classpath, otherwise `InternalResourceView`
- inherited from `UrlBasedViewResolver`: the `redirect:` and `forward:` prefixes

```
return "redirect:/accounts";     // → RedirectView, sends a 302
return "forward:/other";         // → server-side forward, no round trip
```

`BeanNameViewResolver` — Views As Beans

```
<bean class="org.springframework.web.servlet.view.BeanNameViewResolver">
    <property name="order" value="0"/>
</bean>

<bean name="csvReport" class="com.example.CsvView"/>
```

```
<bean name="goHome" class="org.springframework.web.servlet.view.RedirectView">
    <property name="url" value="/home"/>
    <property name="contextRelative" value="true"/>
</bean>
```

Now `return "csvReport";` renders your `CsvView`, and `return "goHome";` redirects.

It returns `null` when no bean of that name exists, so it plays nicely in a chain. Put it **before** `InternalResourceViewResolver` and you get: named beans win, everything else falls through to JSP.

`ResourceBundleViewResolver` and `XmlViewResolver`

Both **deprecated since Spring 5.3** — still present in 6.x, so you will meet them in older projects.

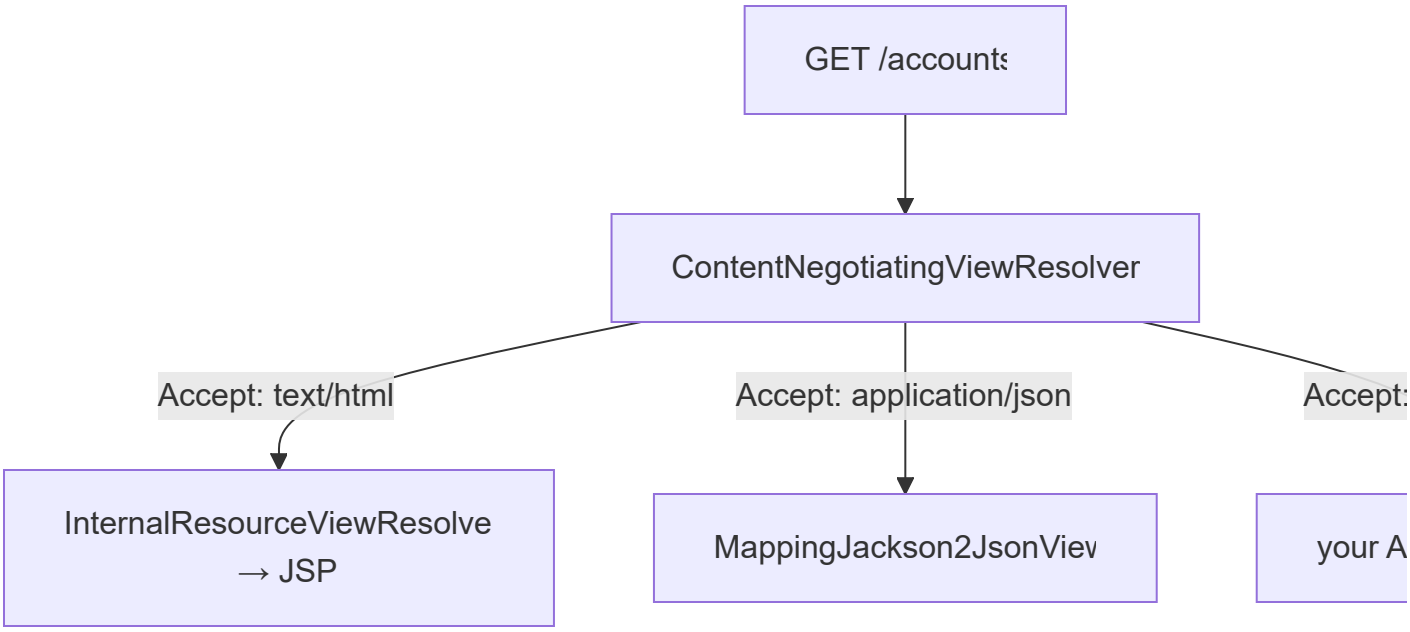
`ResourceBundleViewResolver` reads `views.properties` from the classpath:

```
account.(class)=org.springframework.web.servlet.view.JstlView
account.url=/WEB-INF/views/account.jsp
```

`XmlViewResolver` reads the same idea out of `/WEB-INF/views.xml` as bean definitions.

Both were ways to declare views externally before `BeanNameViewResolver` and plain `@Bean` methods made it unnecessary. The deprecation note says exactly that: *use the common view resolver variants instead.* If you see them, the migration target is `BeanNameViewResolver` or `InternalResourceViewResolver`.

`ContentNegotiatingViewResolver` — One URL, Many Formats



It does not resolve anything itself. It asks the other resolvers, collects the candidate `View`s, and picks by the requested media type. A resolver whose whole job is delegating to resolvers — which only works because they all share one interface.

The `View` Implementations

View	Renders
`InternalResourceView` / `JstlView`	forwards to a JSP
`RedirectView`	sends a 302, optionally carrying flash attributes
`MappingJackson2JsonView`	the model serialised as JSON
`MarshallingView`	the model as XML
`AbstractPdfView`	a PDF you build in `buildPdfDocument`
`AbstractXlsxView`	an Excel workbook
`FreeMarkerView`, `ThymeleafView`	template engines

All of them implement one two-method interface. A PDF and a JSP forward are the same thing to `DispatcherServlet`.

The Model Becomes Request Attributes

This is the join between the two halves of MVC, and it is much simpler than it looks.

```
mav.addObject("shape", "AbstractController");     // in the controller
```

```
request.setAttribute("shape", "AbstractController");   // what render() does
```

```
<p>Handler shape: ${shape}</p>                        <!-- in the JSP -->
```

`${shape}` is **Expression Language**. It searches four scopes in order — page, request, session, application — and the model landed in **request** scope. That is the whole mechanism. No magic, no binding, no Spring involvement at render time.

Two consequences:

- A model attribute lives for **one request only**. Refresh the page and it is rebuilt.
- `${user.name}` calls `getName()` on the object. EL walks JavaBean properties, so your model objects need getters.

Why Views Live Under `/WEB-INF`

```
<property name="prefix" value="/WEB-INF/views/"/>
```

The servlet specification says the container must **never serve anything under `/WEB-INF` directly** to a browser. A forward from inside the application is still allowed.

So `/WEB-INF/views/home.jsp` can be rendered by a controller, but typing that URL gives a 404.

That matters because a JSP reached directly would run **with an empty model** — every `${...}` would be blank, and any page expecting data would break or leak a half-rendered layout. Putting views under `/WEB-INF` makes "you must go through a controller" a rule the container enforces rather than a convention you hope people follow.

The View Layer Is Replaceable — Here Is the Proof

Same controller, same model, three different view technologies. Nothing in the controller changes:

Return this	Resolver that answers	What the user gets
`"home"`	`InternalResourceViewResolver`	a forward to a JSP
`"plainTextView"`	`BeanNameViewResolver`	whatever your `View` bean writes
`"goHome"`	`BeanNameViewResolver`	a `RedirectView` → HTTP 302
`"redirect:/home"`	`UrlBasedViewResolver` prefix	the same 302, without a bean

The controller returns a `String` in all four cases and knows nothing about which happened. That is the entire argument for having seam 4 at all — and the demo project has one route for each row.

Chaining Resolvers

`DispatcherServlet` holds a `List<ViewResolver>`, sorted by `Ordered`, and takes the first non-`null`:

```
for (ViewResolver resolver : this.viewResolvers) {
    View view = resolver.resolveViewName(viewName, locale);
    if (view != null) {
        return view;
    }
}
```

Same chain-of-responsibility shape as `HandlerMapping`. A sane chain:

Order	Resolver
`0`	`BeanNameViewResolver` — named `View` beans win
`1`	`ContentNegotiatingViewResolver` — format-specific
`100`	`InternalResourceViewResolver` — the catch-all, **must be last**

Put `InternalResourceViewResolver` anywhere but last and nothing after it will ever run.

Why It Is Structured Like This

Three Patterns, Stacked

Pattern	Where	What it buys
Strategy	all four seams	swap an implementation by changing configuration, not code
Adapter	seam 2 specifically	invoke handlers whose signatures are genuinely incompatible
Chain of responsibility	mapping list, resolver list	several implementations coexist; first match wins
Template Method	`AbstractController`, `AbstractHandlerMapping`	inherit the plumbing, override one method

And the reason for all of it is the **Open/Closed Principle**: `DispatcherServlet` is open to extension by adding beans, closed to modification. You never subclass it. You never edit it. In twenty years, essentially nobody has needed to.

The Proof It Worked

Year	New handler style	What Spring had to add	What changed in `DispatcherServlet`
2004	`Controller` interface	—	—
2007	`@Controller` + `@RequestMapping`	a mapping + an adapter	**nothing**
2011	`HandlerMethod` infrastructure	a better mapping + adapter	**nothing**

Four generations of controller style. The orchestrator was written once.

This is also why old applications keep running: `SimpleControllerHandlerAdapter` is still registered by default in Spring 6, so a `Controller` written in 2005 still works today next to an `@RestController` written this morning.

Cheat Sheet — Who Invokes What

Handler you write	Mapping that finds it	Adapter that invoke
`@Controller` + `@GetMapping`	`RequestMappingHandlerMapping`	`RequestMappingHand`
`@RestController`	`RequestMappingHandlerMapping`	`RequestMappingHand`
`implements Controller`	`BeanNameUrlHandlerMapping` or `SimpleUrlHandlerMapping`	`SimpleControllerHa`
`extends AbstractController`	`SimpleUrlHandlerMapping`	`SimpleControllerHa`
`ParameterizableViewController`	`SimpleUrlHandlerMapping`	`SimpleControllerHa`
`implements HttpRequestHandler`	`SimpleUrlHandlerMapping`	`HttpRequestHandler`
your own handler type	whichever you configure	**your own adapter**

Demo Project — `springmvc-four-components`

One `DispatcherServlet`, one XML file, every seam visible at once.

No `@EnableWebMvc`, no `<mvc:annotation-driven/>`, no component scan, no interceptors. Every strategy object is a bean you can point at in `/WEB-INF/dispatcher-servlet.xml` — **that file is the deliverable**; the Java and the JSPs are deliberately as thin as they can be.

Three mappings competing in one ordered chain · **four adapters**, one of them ours · **six handler shapes** · **two view resolvers** chained by order.

```
mvn clean package cargo:run
→ http://localhost:8080/fourcomp/
```

The Routes, And What Each One Isolates

URL	Handler shape	Mapping	Ada
`/diagnostics`	`AbstractController`	`SimpleUrlHandlerMapping`	`Sim`
`/home`	`AbstractController`	`SimpleUrlHandlerMapping`	`Sim`
`/about`	`ParameterizableViewController`	`SimpleUrlHandlerMapping`	`Sim`
`/legacy`	`Controller` interface	`BeanNameUrlHandlerMapping`	`Sim`
`/annotated`	`@Controller`	`RequestMappingHandlerMapping`	`Req`
`/raw`	`HttpRequestHandler`	`SimpleUrlHandlerMapping`	`Htt`

URL	Handler shape	Mapping	Ada
`/greet`	**our own type**	`SimpleUrlHandlerMapping`	**our**
`/plain`	`AbstractController`	`SimpleUrlHandlerMapping`	`Sim`
`/go-home`	`ParameterizableViewController`	`SimpleUrlHandlerMapping`	`Sim`

Each row changes **one** thing against the row above it. Read them in order.

Session 1 Recap

- **MVC** is a pattern; **Spring MVC** is Spring's implementation of it, built on servlets
- `DispatcherServlet` is the Front Controller — one entry point, delegates everything
- **Root context** (services) is the **parent** of the **Web context** (controllers); child sees parent, not vice versa
- `HandlerMapping` = *which* handler (+ interceptors, as a `HandlerExecutionChain`)
- `HandlerAdapter` = *how* to invoke that handler — the **Adapter pattern**, because handlers come in several shapes
- **Controllers**: `@Controller` (modern), `Controller` / `AbstractController` (legacy), `HttpRequestHandler` — three shapes, three adapters
- **Model / ModelMap / ModelAndView** carry data to the view
- `ViewResolver` = logical name → real `View`
- If you define no beans, `DispatcherServlet.properties` supplies **defaults**; `@EnableWebMvc` upgrades them